

1 TUTORIAL DE SIMULAÇÃO (VHDL) - MIPS_S_withBRAMS

Este tutorial visa iniciar os alunos à simulação do sistema MIPS_S_withBRAMs, que inclui um processador que dá suporte a um subconjunto de instruções da arquitetura MIPS, denominado MIPS_S. Este processador implementa uma organização que pode executar um subconjunto das instruções da arquitetura MIPS32 (dando suporte a executar apenas 37 instruções desta arquitetura). Uma especificação detalhada do processador do MIPS_S_withBRAMs está disponível (**Spec_ArqOrg_MIPS_S_V3.01.pdf**, no diretório **Distribution/0_Docs**)¹. As principais instruções da arquitetura MIPS32 às quais a arquitetura MIPS_S **não dá suporte** são as instruções de manipulação de números de ponto flutuante, as instruções de acesso à memória por meias palavras e as de acesso desalinhado à memória. O acesso a byte é possível usando instruções **lbu** e **sb**.

Como o que se pretende é prototipar o sistema em FPGAs de plataformas físicas reais, ao processador acrescentou-se memórias de instruções e dados baseadas em estruturas de hardware disponíveis internamente em FPGAs da AMD/Xilinx, conforme detalhado abaixo. Nos modelos de simulação de processadores, um *testbench* complexo permite carregar memórias (tipicamente não-sintetizáveis) com um programa qualquer (que use o subconjunto de instruções ao qual a MIPS_S dá suporte) e seus dados, isto no início da simulação. Em relação à abordagem de simulação, se procede aqui de forma diferente, usando um processo que sintetiza um sistema processador-memórias pronto para executar um programa específico sobre dados específicos. Uma desvantagem desta abordagem é que para cada novo conjunto programa/dados a executar, o sistema deve ser totalmente (re-)sintetizado, usando, por exemplo, o ambiente ISE ou Vivado da AMD/Xilinx. Aqui apresenta-se apenas a estrutura geral da descrição sintetizável e procede-se a alguns exercícios de simulação para levar os alunos a dominarem a estrutura do sistema MIPS_S withBRAMs.

1.1 Simulação do sistema MIPS_S_withBRAMs

O ambiente de simulação/síntese a ser usado aqui encapsula o processador MIPS_S e duas memórias (de instruções e dados) necessárias à execução de programas por este processador. A organização do sistema é ilustrada na Figura 1. As portas da interface externa, além do **clock** e do **reset**, são sinais que permitem a entidades externas ler/escrever os conteúdos da memória de dados e eventualmente interagir com o processador usando operações de entrada/saída mapeadas no mapa de memória do MIPS_S.

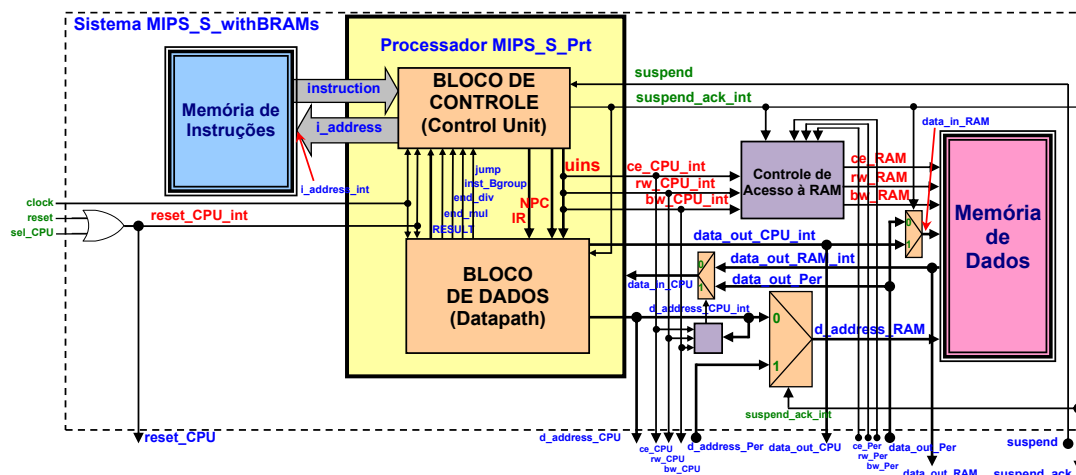


Figura 1 – Diagrama de blocos do sistema MIPS_S_withBRAMs contendo o processador MIPS_S, memórias de instruções e dados e uma lógica de cola entre estes. O processador MIPS_S em si (retângulo amarelo e seus conteúdos) conecta-se às memórias de instruções e de dados. Alguns sinais ou nomes de sinais podem ter sido omitidos na Figura, para beneficiar a clareza.

A Figura 2 mostra uma visão parcial do código VHDL do ambiente. Este contém as instâncias dos seus 3 blocos principais: a Memória de Instruções, a Memória de Dados, e o processador MIPS S Prt,

¹ Note que a organização MIPS_S usada aqui e aquela descrita na especificação em anexo são levemente distintas. As principal diferença é que A MIPS_S descrita na especificação é voltada para simulação, sobretudo (embora ela seja sintetizável). A MIPS_S usada aqui foi adaptada para lidar eficientemente com os blocos de memórias de FPGAs da AMD/Xilinx (denominados BRAMs) que podem ser configurados com duas portas de acesso separadas. Assim, o processador teve sua interface externa alterada para usar dois conjuntos de pinos de acesso à memória de dados: 32 fios de saída para escrita de um dado na memória (sinal **data out**) e 32 fios de entrada para leitura da memória (sinal **data in**).

acrescidos de uma “lógica de cola” (*glue-logic*) simples para permitir o controle da comunicação entre as diversas partes do ambiente e sinais de interface do sistema MIPS_S_withBRAMs completo.

```

library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.Numeric_std.all;
--use IEEE.Std_Logic_unsigned.all;
use work.p_MIPS_S.all;

entity MIPS_S_withBRAMs is
  generic(LAST_I_ADDRESS : wires32:=x"004007FF";
    FIRST_D_ADDRESS : wires32:=x"10010000"; LAST_D_ADDRESS : wires32:=x"10013FFF");
  port(clock, reset, sel_CPU: in std_logic;
    reset_CPU: out std_logic;
    suspend, ce_Per, rw_Per, bw_Per: in std_logic;
    d_address_Per, data_out_Per: in wires32;
    suspend_ack, ce_CPU, rw_CPU, bw_CPU: out std_logic;
    d_address_CPU, data_out_CPU, data_out_RAM: out wires32);
end MIPS_S_withBRAMs;

architecture MIPS_S_withBRAMs of MIPS_S_withBRAMs is
  ...
  begin
    reset_CPU_int <= reset or sel_CPU; reset_CPU <= reset_CPU_int;
    Mem_Clock: IDMem_clock <= not clock;
    i_address_int <= i_address(10 downto 2);

    P_mem: entity work.program_memory port map(clock=>IDMem_clock,
      address=>i_address_int, instruction=>instruction);

    I_MIPS_S: entity work.MIPS_S port map (clock=>clock, reset=>reset_CPU_int,
      ce=>ce_CPU_int, rw=>rw_CPU_int, bw=>bw_CPU_int, suspend=>suspend,
      suspend_ack=>suspend_ack_int, i_address=>i_address,
      instruction=>instruction, d_address=>d_address_CPU_int,
      data_in=>data_in_CPU, data_out=>data_out_CPU_int);

    I_Ctrl_Sigs: valid_d_address <= '1' when (suspend_ack_int='0' and
      (d_address_CPU_int >= FIRST_D_ADDRESS and d_address_CPU_int <= LAST_D_ADDRESS))
      or (suspend_ack_int='1' and (d_address_Per >= FIRST_D_ADDRESS
      and d_address_Per <= LAST_D_ADDRESS)) else '0';
    ce_RAM <= ce_CPU_int when (valid_d_address='1' and suspend_ack_int='0')
      else ce_Per when (valid_d_address='1' and suspend_ack_int='1') else '0';
    rw_RAM <= rw_CPU_int when (valid_d_address='1' and suspend_ack_int='0')
      else rw_Per when (valid_d_address='1' and suspend_ack_int='1') else '1';
    bw_RAM <= bw_CPU_int when (valid_d_address='1' and suspend_ack_int='0')
      else bw_Per when (valid_d_address='1' and suspend_ack_int='1') else '0';
    D_RAM_Addr_Mux: d_address_RAM <=
      d_address_CPU_int(13 downto 0) when (suspend_ack_int='0')
      else d_address_Per(13 downto 0);
    D_RAM_Data_Mux_in: data_in_RAM <= data_out_CPU_int when (suspend_ack_int='0')
      else data_out_Per;
    CPU_Data_Mux_in: data_in_CPU <= data_out_Per when (ce_CPU_int='1' and
      rw_CPU_int='1' and (not (d_address_CPU_int >= FIRST_D_ADDRESS and
      d_address_CPU_int <= LAST_D_ADDRESS))) else data_out_RAM_int;
    D_mem: entity work.Data_Memory port map (clock=>IDMem_clock, ce=>ce_RAM, rw=>rw_RAM,
      bw=>bw_RAM, address=>d_address_RAM(13 downto 0), data_in=>data_in_RAM,
      data_out=>data_out_RAM_int);
    d_address_CPU <= d_address_CPU_int;
    ce_CPU <= ce_CPU_int; rw_CPU <= rw_CPU_int; bw_CPU <= bw_CPU_int;
    data_out_CPU <= data_out_CPU_int; data_out_RAM <= data_out_RAM_int;
    suspend_ack <= suspend_ack_int;
  end MIPS_S_withBRAMs;

```

Figura 2 – Código VHDL (parcial) que implementa o par entidade/arquitetura ilustrado na Figura 1.

1.2 Como as memórias do ambiente são organizadas?

As memórias empregadas neste sistema devem ser sintetizáveis. Uma maneira eficiente de fazer isto é utilizar estruturas especiais existentes dentro de FPGAs para implementar memórias de algum porte

(na ordem das dezenas a milhares de Kbits). Os FPGAs de diversas famílias da AMD/Xilinx (Spartan2, Spartan6, Artix-7, Virtex-7, etc.) possuem certa quantidade de blocos de memória de alta densidade de 16 ou mais Kbits. Estes blocos são configuráveis², e se denominam genericamente de BlockRAMs ou BRAMs. BRAMs foram usadas neste ambiente para implementar os blocos de memória necessários.

Descreve-se a seguir o mapeamento das memórias de instruções e dados para BRAMs em FPGAs da AMD/Xilinx.

1.2.1 Memória de Instruções

O acesso à Memória de Instruções no MIPS é sempre realizado buscando palavras de 32 bits alinhadas em endereços múltiplos de 4, conforme detalhado na especificação da MIPS_S. Assim, ao invés de se utilizar uma organização com acesso a bytes individuais, pode-se usar uma memória organizada de forma simples apenas com acesso a palavras. No caso do ambiente optou-se então por utilizar uma única BRAM para instruções, configurando seus 16Kbits como uma memória de 512 palavras de 32 bits. **Note-se que isto limita os programas executados pelo processador a não terem mais de 512 instruções de código objeto, o que é mais que suficiente para os fins didáticos a que se destina o ambiente. Estes valores podem ser aumentados sem muita dificuldade.**

Esta memória usa uma codificação específica em VHDL, baseada em blocos pré-definidos de um par biblioteca/pacote fornecidos pela AMD/Xilinx (no caso, a biblioteca UNISIM, e o pacote vcomponents). O componente adequado para usar aqui é denominado RAMB16_S36³. Para usar tal componente, basta declarar a biblioteca UNISIM e o pacote vcomponents no código VHDL que usa a memória e instanciar esta. Um exemplo é mostrado na Figura 3.

```

programa: RAMB16_S36 generic map
(
    INIT_00 => X"8e520000343200083c0110018e310000343100043c011001343d08003c011000",
    INIT_01 => X"afb3000cafb20008afb10004afbf000027bdfbf08e7300003433000c3c011001",
    INIT_02 => X"342800003c01100127bd00108fb400048fbf00000100f809342800883c010040",
    INIT_03 => X"afa90008afa80004afbf000027bdfbf48fa900088fa8000403e00008ad140000",
    INIT_04 => X"27bdfbf48fa9000c27bd000c8fa800048fbf00000100f809342800ec3c010040",
    ...
    INIT_3D => X"0000000000000000000000000000000000000000000000000000000000000000",
    INIT_3E => X"0000000000000000000000000000000000000000000000000000000000000000",
    INIT_3F => X"0000000000000000000000000000000000000000000000000000000000000000"
)
port map
(
    CLK  => clock,
    ADDR => address,
    EN   => '1',
    WE   => '0',
    DI   => x"00000000",
    DIP  => x"0",
    DO   => instruction,
    SSR  => '0'
);

```

Figura 3 – Exemplo de estrutura parcial de declaração, inicialização e instanciação de memórias específicas para FPGAs AMD/Xilinx. No caso, apresenta-se uma instância da Memória de Instruções para a MIPS_S_withBRAMs.

² Na realidade, BRAMs são blocos de 18Kbits, pois para cada *byte* (8 bits) desta memória existe 1 bit de paridade, para fins de detecção de erros de leitura/escrita. Quando se ignora os bits de paridade (o que é feito aqui), tem-se uma memória de 16Kbits. Para famílias de FPGAs mais recentes, existem BRAMs maiores, de 32/36Kbits, que não serão discutidas ou usadas aqui.

³ A nomenclatura RAMB16_S36 vem da AMD/Xilinx, e pode se revelar um pouco confusa para neófitos. Neste caso, o primeiro numeral (16) significa que o bloco de 18 Kbits está configurado sem acesso a paridade (portanto disponibilizando apenas 16 Kbits dos 18 Kbits de memória existentes). O segundo numeral (36) significa que cada leitura/escrita de dado da/ná memória faz acesso a 36 bits. Descontando-se os 4 bits de paridade (devido ao fato de não se usar esta, como visto na interpretação do primeiro numeral), sabe-se que a porta de acesso é de 32 bits. BRAMs são memórias de porta dupla. Neste caso, a existência de apenas 1 parâmetro após o caractere sublinhado () indica que apenas uma porta é usada. Outro exemplo de configuração seria RAMB16_S8_S16, onde não se usa paridade (RAMB16) e se tem duas portas de acesso aos 16Kbits, uma de 8 bits de dados (S8) e uma de 16 bits de dados (S16).

A inicialização da memória de instruções com o programa a executar é feita usando parâmetros genéricos INIT, declarados via comando VHDL **generic map**, conforme mostra a Figura 3. O preenchimento em tempo de elaboração do hardware destes blocos pressupõe que cada linha **INIT_xx** desta codificação descreve como preencher 8 palavras consecutivas de 32 bits (ou seja, 4 bytes) de memória. Assim, cada linha especifica o conteúdo de 256 bits (32 bytes ou 8 palavras de 32 bits) da memória, perfazendo um total de 64 linhas (em hexa, 64=0x3F) por BRAM (de 16.384 bits, 64 linhas×256 bits).

O comando **port map** da Figura 3 conecta pinos da memória (pré-definidos pela organização das BRAMS nos FPGAs) a sinais do processador MIPS_S_withBRAMs. Note que no exemplo da Figura 3, o sinal **WE** (habilitação de escrita ou *write-enable*) é conectado à constante '0', o que caracteriza esta memória como apenas de leitura (ROM).

1.2.2 Memória de Dados

A(s) Memória(s) de Dados tem(têm) estrutura distinta da memória de instruções, pois deve(m) possibilitar não apenas a leitura de valores, mas também a escrita em palavras ou bytes isolados (genericamente, deveriam permitir também o acesso em meias palavras, de 16 bits, mas como a MIPS_S não possui instruções para esta forma de acesso hoje, esta flexibilidade na arquitetura da memória é deixada aqui como trabalho futuro), como necessário ao executar a instrução **SB**, por exemplo. Para tanto, utiliza-se uma arquitetura de memória entrelaçada com 4 BRAMs de memória configuradas cada uma com acesso a *byte* e por fora das BRAMs acrescenta-se lógica de cola para permitir o acesso paralelo a um *byte* de cada bloco em um dado instante. Cada bloco é configurado como uma memória de 2.048 palavras de 8 bits. Assim, existe uma memória de dados de 2.048 palavras de 32 bits ($2.048 \times 32 = 65.536$ bits) ou 64Kbits ou 8Kbytes, ou 2Kpalavras de 32 bits por bloco. O componente adequado da biblioteca UNISIM, no pacote vcomponents a usar aqui é denominado RAMB16_S9. Neste documento trabalha-se com um único bloco de Memória de Dados, mas podem existir mais.

1.3 Como se gera o conteúdo para as linhas INIT_XX?

O ponto de partida é um *dump* de memória gerado por um programa montador como o MARS. No MARS, após carregar o arquivo fonte .asm e montá-lo, deve-se usar a opção de menu **File → Dump Memory**, escolhendo fazer *dumps* dos segmentos de texto e de dados (opções .text e .data na janela de diálogo de *dumps*), usando o formato de *dump* Text/Data Segment Window. Geram-se com isto 2 arquivos. Exemplos de nomes para estes são: *prog.txt* e *data.txt*.

O exemplo da Figura 4 corresponde ao *dump* do segmento de texto (segmento .text) do programa em linguagem de montagem do MIPS que gerou o conteúdo usado para produzir a BRAM da Memória de Instruções da Figura 3. Mostra-se apenas a parte inicial do *dump*, que gerou as linhas INIT_00 e INIT_01 na Figura 3. Note-se que o *dump* da área de dados é um pouco diferente, pois usa um formato de texto distinto. Este procedimento de inicialização é propenso a erros, se feito de forma manual. Por isto, são disponibilizados os seguintes recursos no diretório MARS do material de apoio:

[0x00400020]	0x3c011000	Uma linha de INIT contém 32 bytes, ou 8 palavras de 32 bits, ordenadas do endereço maior (à esquerda) para o endereço menor (à direita). Assim, estas 8 linhas de código geram a seguinte linha de INIT: 8e520000343200083c0110018e310000343100043c011001343d08003c011000
[0x00400024]	0x343d0800	
[0x00400028]	0x3c011001	
[0x0040002c]	0x34310004	
[0x00400030]	0x8e310000	
[0x00400034]	0x3c011001	
[0x00400038]	0x34320008	
[0x0040003c]	0x8e520000	
[0x00400040]	0x3c011001	
[0x00400044]	0x3433000c	
[0x00400048]	0x8e730000	
[0x0040004c]	0x27bdf0ff	
[0x00400050]	0xafbf0000	
[0x00400054]	0xafb10004	
[0x00400058]	0xafb20008	
[0x0040005c]	0xafb3000c	

Figura 4 – Exemplo de trecho de arquivo de *dump* usado para gerar o arquivo de inicialização das memórias do processador MIPS_S_withBRAMs.

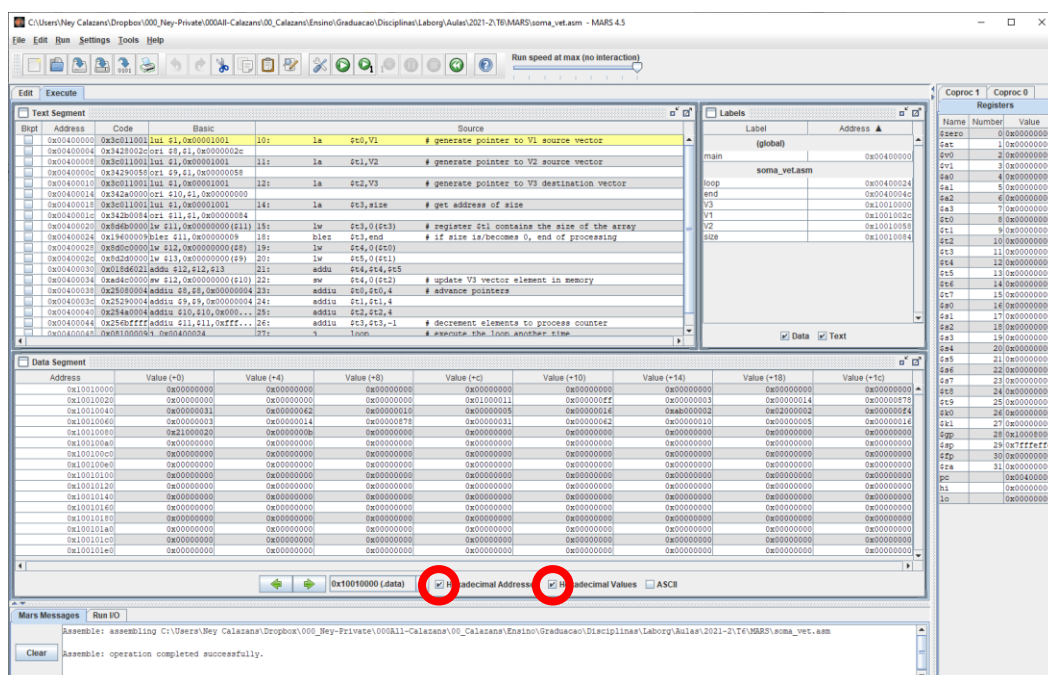
1. **le_mars.c**– código fonte C de um programa que converte os dois arquivos de *dump* para um arquivo VHDL de nome padrão *memory.vhd*. É tarefa do aluno gerar o executável do *le_mars* a partir do código C para o sistema operacional usado. Por exemplo, em um computador com um compilador da linguagem C da GNU instalado, um comando útil é em um *shell* de comandos digitar “gcc -Wall -o le_mars le_mars.c”. O exemplo é baseado em ambientes Linux, mas pode ser definido de forma similar em ambientes Windows e/ou MacOs. Para quem prefere usar um ambiente dotado de interface gráfica, pode-se instalar e usar sistemas gratuitos (*open-source*) de desenvolvimento de programas escritos em C como o Code::Blocks (<https://www.codeblocks.org/>). Depois de compilar o programa e gerar seu código executável, pode-se usar em um ambiente Linux o seguinte comando (em um shell de comandos): *./le_mars prog.txt data.txt* (com o comando *./le_mars* sem parâmetro nenhum obtém-se um texto de ajuda). A execução da primeira linha produz como saída um arquivo de nome **memory.vhd**, que possui como conteúdo um texto válido na linguagem VHDL que implementa as memórias de instrução e dados, pré-carregadas com o código objeto do programa e seus dados.
2. **SomaVet.asm** – arquivo fonte com um programa exemplo em linguagem de montagem do MIPS. No material disponibilizado há outros exemplos de programas válidos em linguagem de montagem (*assembly language*) do MIPS.

2 Tarefas Acessórias

O objetivo desta Seção é prover um treinamento básico em como gerar o código objeto de um programa qualquer do MIPS e preparar os arquivos necessários para realizar uma simulação VHDL do MIPS_S_withBRAMs executando tal programa.

1. Abra o arquivo **SomaVet.asm** no simulador MARS e monte seu código objeto (tecla **F3** dentro do MARS). Note que o programa termina com uma instrução de salto para ela mesma, uma forma de virtualmente “travar” a simulação após o término da execução das tarefas úteis do programa. Use a opção de menu **File → Dump Memory**, escolhendo fazer *dump* dos segmentos de texto (**.text**) e de dados (**.data**) em dois arquivos distintos, por exemplo *prog.txt* e *data.txt*, usando o formato de *dump* **Text/Data Segment Window**.

Atenção: Para que o *dump* de memória do segmento de dados seja produzido na forma esperada pelo software que vai tratá-lo, é necessário que endereços e dados estejam mostrados na tela em hexadecimal, veja os boxes circundados na Figura abaixo:



2. Execute o programa **le_mars.exe** no mesmo diretório onde foram gravados os arquivos de *dump*. A execução do **le_mars.exe** gera o arquivo VHDL **memory.vhd**, contendo o código objeto mapeado nas memórias de instruções e de dados. Esta execução deve ser feita em uma janela textual de comandos, seja no Windows, seja no Linux.
3. Crie um projeto no ISE ou no Vivado que depois usará o ISIM (o simulador do ISE ou do Vivado) para simulação, adicionando os seguintes arquivos:
 - i. **memory.vhd** (gerado no passo 2 acima),
 - ii. **mult_div.vhd** (presente no diretório **Distribution/3_MIPS_S_Prt/VHDL**),
 - iii. **MIPS_S_withBRAMs.vhd** (presente no diretório **Distribution/3_MIPS_S_Prt/VHDL**), e
 - iv. **MIPS_S_sem_perif_tb.vhd** (presente no diretório **Distribution/3_MIPS_S_Prt/VHDL**).

Observação: Caso a biblioteca UNISIM não esteja disponível, contacte o professor para resolver o problema. Isto não deve ocorrer, em geral.

4. Realize a simulação VHDL do sistema criado por um período de 5.15 μ s (ao usar um clock de 100Mhz, o dobro deste valor para um clock de 50MHz). Isto pode ser feito no ISE ou Vivado lançando a simulação do testbench **MIPS_S_sem_perif_tb.vhd** e executando o seguinte comando na Tcl Console da simulação: *restart; run 5.15us*. Em uma janela do tipo forma de onda inserir os sinais conforme mostrado na Figura 5 (Pode-se carregar o arquivo que define esta disposição de formas de ondas, **SomaVet_wform.wcfg**, disponível no diretório denominado **Distribution/3_MIPS_S_Prt/Sw_Wcfg_files**). Note que se trata de uma simulação completa do programa (**SomaVet.asm**) que soma de dois vetores (V1 e V2) de 11 inteiros e escreve o resultado em um terceiro vetor de mesmo tamanho (V3). Algumas das informações perceptíveis na Figura são:

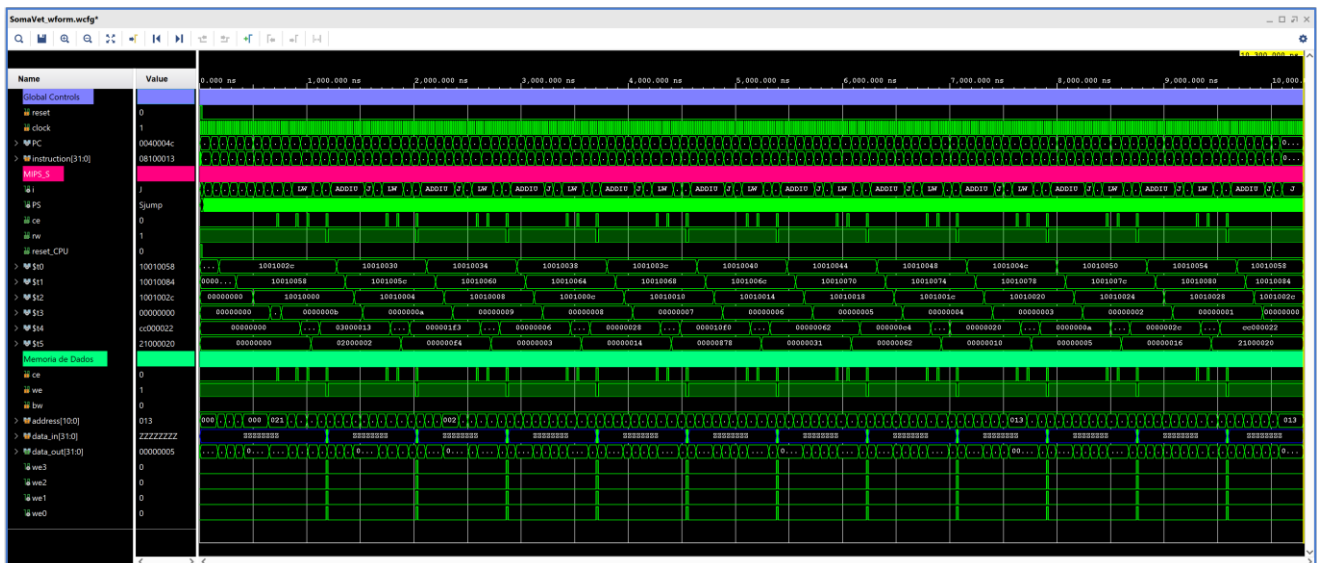


Figura 5 – Simulação completa da execução do programa **SomaVet.asm** no processador **MIPS_S_withBRAMs**.

- A. O tempo de simulação do programa completo (cerca de 5,15 μ s), assumindo o uso de um clock de 100MHz. Para um clock de 50MHz, altere (se necessário) a definição do sinal de relógio no testbench e simule pelo dobro do tempo (10,3 μ s);
- B. As mudanças nos registradores \$t3 (Reg(11) na forma de onda), \$t4 (Reg(12) na forma de onda) e \$t5 (Reg(13) na forma de onda), que contém respectivamente **o contador de elementos processados** que vai decrescendo de 11 (0xB) a 0, **o resultado da soma dos dois elementos correntes** de V1 e V2 e **o elemento corrente** de V2;
- C. Os momentos de escrita na memória, quando os sinais do processador ce e rw forem ce='1' e rw='0'. Isto ocorre exatamente 11 vezes na Figura (os vetores somados têm 11 elementos neste exemplo).